The Maze Agent

A typed agent solving a problem over a CLI, on a free model

October 30, 2026

Abstract

This note describes an addition to the `starship` showpiece: a single agent that solves a maze. The agent perceives a grid, fights the monsters that block its path, and reaches an exit. It runs in one of two modes. In the default mode a deterministic search supplies every move, so the demo needs no network and replays to the same bytes twice. In the other mode a free language model supplies every move through the same loop and the same gate. Both modes have been run against this maze, the language-model mode on Groq with `openai/gpt-oss-120b`, and both reach the exit. The document covers five things in order: the world the agent lives in, the command-line surface it acts through, the harness that runs the agent and logs it, the container orchestration layer the agent plugs into, and the single flag that switches between the deterministic solver and the language model.

Contents

1 What was built, and why **2**

2 The world **2**

3 The command-line surface **3**

4 The harness **4**
- 4.1 The model seam . . . 4
- 4.2 The reason-gate and the two logs . . . 5
- 4.3 The loop . . . 5

5 The orchestration layer **6**
- 5.1 Orchestration of a single worker: the loop . . . 6
- 5.2 Orchestration of many workers: the container combinators . . . 6
- 5.3 Where the maze stands . . . 6
- 5.4 The planned composition . . . 7

6 How the harness meets the orchestration layer **7**

7 One flag: deterministic or language model **8**
- 7.1 Deterministic mode . . . 8
- 7.2 Language-model mode . . . 8

8 Running and checking it **9**

1 What was built, and why

The `starship` showpiece already stands up four of the five parts of a classic agent harness: a client, a conversation slice, an outer user loop, and a tool-dispatch seam. Its officers are scripted stand-ins for a model, so the fifth part, the inner model-call loop, was never present. Two needs drove this work. The first was to run the harness on a model that costs nothing and needs no Anthropic key. The second was a worked example in which an agent *solves* something a reader grasps at a glance, rather than reading a single delegate-decide-act pass.

A grid maze answers both. The goal is unambiguous: reach the exit. Success is checkable: the agent either stands on the exit or it does not. The task exercises perception, a decision, an action, and a repeat, which is the shape of the inner loop. The monsters give the gate something to refuse. The whole run finishes in about a dozen steps.

Everything below lives in the `starship` repository under `src/`. The maze world is `src/maze/`, its command-line surface is `src/cli/dungeon.ts`, and the agent is `src/agent/`. The harness gate, the two logs, and the container combinators are reused unchanged from the star-system half of the repository.

2 The world

The world is plain data. A ship's state in the star system is fuel, hull, and position; a maze is a grid, a hero, some monsters, and an exit. Nothing in the world has behaviour. The reducers that change it live in `src/maze/world.ts` and the smart constructors that decide which changes are legal live in `src/maze/gates.ts`. Holding the state as data is what lets it serialise to one JSON file and replay to the same bytes.

```
export type Pos = { readonly x: number; readonly y: number };
export type Dir = "N" | "S" | "E" | "W";

export type Monster = {
  readonly id: string;
  readonly pos: Pos;
  readonly hp: number;
  readonly atk: number;
};

export type MazeWorld = {
  readonly width: number;
  readonly height: number;
  readonly walls: readonly string[]; // rows of '#' (wall) and '.' (floor)
  readonly hero: { readonly pos: Pos; readonly hp: number; readonly atk: number };
  readonly monsters: readonly Monster[];
  readonly exit: Pos;
};
```

Coordinates run with `x` as the column and `y` as the row, with `y` increasing downward. The four moves are therefore $N : y{-}1$, $S : y{+}1$, $E : x{+}1$, $W : x{-}1$. A cell off the grid counts as a wall, so the hero cannot step off the edge. The seeded maze in `src/maze/seed.ts` is a compact snake corridor with one monster on each leg, so there is exactly one route and both monsters must be fought rather than skirted.

```
+-----+
|@.G..|      @  hero      hp 20   (0,0)
|####.|      X  exit              (0,2)
```

```
|X.O..|      G  goblin-1  hp 3    (2,0)   atk 1
+-----+      O  orc-1     hp 5    (2,2)   atk 2
```

A change to the world is a tagged value called an effect. The maze has two.

```
export type MazeEffect =
  | { readonly tag: "move"; readonly to: Pos; readonly dir: Dir }
  | { readonly tag: "attack"; readonly targetId: string;
      readonly monsterHpAfter: number; readonly heroHpAfter: number;
      readonly lethal: boolean };
```

A gate produces an effect; `applyEffect` consumes one. `applyEffect` is a pure total function that returns a new `MazeWorld` and never mutates the old. An attack resolves both blows inside the gate: the hero hits for `hero.atk`, and if the monster survives it strikes back for `monster.atk`. A lethal blow lands before the monster can counter. This keeps `applyEffect` a plain writeback with no combat logic of its own.

The world also answers one read, `look`, which is the agent's only window on its surroundings. It returns the hero's position and hit points, what lies in each of the four directions, and where the exit is.

```
export type Look = {
  readonly hero: Pos & { readonly hp: number };
  readonly onExit: boolean;
  readonly neighbours: readonly {
    readonly dir: Dir;
    readonly cell: "wall" | "floor" | "exit";
    readonly monster: string | null;
  }[];
  readonly exit: Pos;
  readonly exitDistance: number; // Manhattan
  readonly exitBearing: string;  // e.g. "SE"
};
```

3 The command-line surface

The agent never touches the world object. It acts through one executable, `src/cli/dungeon.ts`, which is a second copy of the star-system conn (`src/cli/con.ts`) over the maze. Every capability is a subcommand. Read commands are free. Mutating commands require a reason and pass through a gate before anything commits.

Command	Kind	Returns / effect
`look`	read	neighbours, hero state, exit bearing
`hero`	read	position, hp, attack, alive
`map`	read	an ASCII map of the whole maze
`log recent | trace`	read	recent ship's-log entries, or one by id
`move --dir N|S|E|W`	mutate	step one cell; gated by `moveGate`
`attack --target <id>`	mutate	strike an adjacent monster; gated by `attackGate`

A gate returns either an `Ok` carrying a legal effect or a `Refused` carrying a reason. A refusal is an ordinary return value, not an exception. A move into a wall, a move off the edge, and a move onto a monster are all refused, the last with the monster's id so the agent knows what to

attack. This is the smart-constructor rule from the rest of `starship`: a denied command and a broken tool look the same from the caller's seat.

```
export function moveGate(world: MazeWorld, dir: string): GateResult {
  if (!isDir(dir)) return refused(`bad --dir ${JSON.stringify(dir)} (want N|S|E|W)`
    );
  const to = step(world.hero.pos, dir);
  if (offGrid(world, to)) return refused(`the edge of the maze is to the ${dir}`);
  if (isWall(world, to))  return refused(`a wall blocks the way ${dir}`);
  const m = monsterAt(world, to);
  if (m) return refused(`${m.id} blocks the way ${dir} - attack it or go around`);
  return ok({ tag: "move", to, dir });
}
```

The mutating path is one closure, `commit`, lifted from `con.ts` without change of shape. It requires a non-empty `--reason`, runs the gate, applies the effect, saves the world, and writes one line to the ship's log with a correlation id. It does not write the reason. The exit code is `0` on success and `1` on any refusal, so a shell caller can branch on it.

"The various agents"

In the star system each officer has a different sub-surface. Science may read but not fire. Helm may steer but not fire. The surface a given agent may use is its registry, and the registry is the whole permission grant. The maze has one agent, the adventurer, whose registry is the six commands above (the two mutations plus the reads). The same mechanism gives one agent many sub-surfaces or many agents one surface each; the maze uses the simplest case.

4 The harness

The harness is the layer that runs the agent and records what it did and why. It has the five parts named by Thorsten Ball's Go agent and by the toy harness: a client, a conversation slice, an outer loop, an inner model-call loop, and tool dispatch. The maze adds nothing to the two gates the star system already enforces; it supplies the inner loop the star system was missing.

4.1 The model seam

The one place a model enters is an interface, `ModelClient`, in `src/agent/model.ts`. The loop and the dispatch speak only the neutral types below. A provider is the only thing that knows a wire format, so the dispatch stays independent of any model SDK and can be tested with no key and no network.

```
export type ToolCall   = { id: string; name: string; input: Record<string, unknown>
    };
export type ToolResult = { toolUseId: string; content: string; isError: boolean };
export type Turn       = { text: string; toolCalls: readonly ToolCall[] };

export interface ModelClient {
  readonly name: string;
  converse(system: string, messages: readonly Message[],
           tools: readonly ToolSpec[]): Promise<Turn>;
}
```

A `ToolCall` is the model's request to run a capability. A `ToolResult` is what the harness hands back. Swapping the provider changes nothing above this line.

4.2 The reason-gate and the two logs

A tool call becomes a read or a mutation. The dispatch, in `src/agent/loop.ts`, sends a read through `conRead` and a mutation through `runMutation`, both from `src/bridge/harness.ts`. `runMutation` enforces two gates before the command reaches the conn. The first is the registry: a capability not in the allowlist cannot be invoked. The second is the reason: a mutating call with an empty reason is refused, and the refusal is recorded. When both gates pass, `runMutation` appends an `attempted` record to the intent-log, runs the conn with `--reason` and `--corr`, then appends the outcome.

Two logs result. The ship's log (`src/log/shiplog.ts`) records what committed, with a correlation id, written by the conn. The intent-log (`src/log/intentlog.ts`) records why, written by the harness, write-ahead. They join on the correlation id. Reconciliation (`src/log/reconcile.ts`) cross-checks them: a committed mutation with no logged reason is an alarm, and a refused mutation appears only in the intent-log, which is information the ship's log alone could never show.

4.3 The loop

The inner loop is `runMaze` in `src/agent/loop.ts`. It calls the model, services every tool call the model asks for, feeds the results back, and repeats. It stops when the agent stands on the exit, when the hero dies, or when a step budget is reached. The step budget bounds a model that loops.

```
while (true) {
  const world = loadMaze(worldPath);
  if (solved(world) || dead(world)) break;
  if (steps >= maxSteps || turns >= maxTurns) break;

  const turn = await client.converse(SYSTEM_PROMPT, messages, ADVENTURER_TOOLS);
  turns++;
  messages.push({ role: "assistant", text: turn.text, toolCalls: turn.toolCalls });

  const results: ToolResult[] = [];
  for (const call of turn.toolCalls) {
    const result = dispatch(ctx, reg, call);      // through the reason-gate
    results.push(result);
    if (call.name === "move" || call.name === "attack") steps++;
  }
  messages.push({ role: "tool", results });
}
```

A refusal is not thrown. It returns to the model as an error result, so the agent can read "a wall blocks the way N", correct itself, and try again. The agent's only hands are the five allowlisted capabilities. There is no `bash` and no generic "run a command" tool.

```
ModelClient (groq | scripted)
    ↓ Turn { text, toolCalls }
runMaze   (the inner loop)
    ↓ dispatch(call)
reason-gate   (registry check, reason check, write-ahead)
    ↓ conRead / runMutation
dungeon CLI   (gate → applyEffect → save → ship's log)
    ↓ ToolResult { content, isError }
back up to the model, and reconcile(ship's log, intent-log) at the end
```

5 The orchestration layer

Orchestration means two different things in this repository, and the maze uses one of them today.

5.1 Orchestration of a single worker: the loop

MacBeth's note *The toy harness is a worker* reads a harness as a container morphism $\Delta S \otimes X \to Y$: the transcript is the state S, one turn is a step on it, and `run_agent` is iteration of that step. The maze's `runMaze` is exactly this iteration over one worker. In this sense the loop *is* the orchestration: it drives one agent through many turns and holds the conversation that is its state. The permission gate is a smart constructor on the shapes that worker may be asked for.

5.2 Orchestration of many workers: the container combinators

The other sense is the one the showpiece's tagline means by "delegation and amalgamation". It lives in `src/lib/algebra.ts`, the `gold` container combinators vendored into `starship`. A leaf is a `Runner<C>`: give it a shape, get a position over that shape.

```
export interface Runner<C extends Cont> {
  run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;
}
```

`leafLocal` wraps an answer function in a decoder, so an untyped reply becomes a typed position or the leaf throws. `tensorC`, `sumC`, `productC`, and `seqC` compose runners. The star-system captain (`src/bridge/captain.ts`) uses `tensorC` to ask Science and Engineering at once and join their two replies into one picture. Control flows down as a delegated shape; data flows up as a position. In the star system that join is a gather; in the treasury it is an algebraic sum.

5.3 Where the maze stands

The maze runs a single agent, so in the many-worker sense its orchestration is the trivial case: one leaf, no composition. The showpiece's other half, the bridge, is where delegation and amalgamation are demonstrated. These are two axes of the same framework. The bridge demonstrates composition with scripted leaves; the maze demonstrates the model-in-the-loop those leaves stood in for. The point of separating them is that each is legible on its own.

The tension, and its resolution

The repository's tagline is "orchestrated by delegation and amalgamation", yet the maze delegates to nobody. The resolution is that the maze proves the other axis, the real inner loop, which the bridge never had because its officers were scripted. The practical difference is small: the maze agent is already a leaf. It answers shapes (`look`, `move`, `attack`) and the engine licenses the effects, which is precisely the thin-leaf shape `src/bridge/officers.ts` uses. Composing several such leaves is addition, not rework.

5.4 The planned composition

The natural next step reuses `tensorC` without touching the harness. A read-only *Scout* leaf (`look`, `map`) and a *Fighter* leaf (`attack`) are gathered by an adventurer-captain that decides from the Scout's report and orders the Fighter. The gate and the two logs are unchanged; only a new captain and two thin leaves are added. This is left as an extension so the single-agent loop reads clearly first.

6 How the harness meets the orchestration layer

The contact point is one function, `dispatch` in `src/agent/loop.ts`. The loop (orchestration, in the single-worker sense) produces a `ToolCall`. `dispatch` maps that call to a conn argument vector and routes it through the harness gate.

```
export function dispatch(ctx: HarnessCtx, reg: Registry, call: ToolCall):
    ToolResult {
  const plan = planCall(call);                  // name -> argv, mutates?, reason
  if (!plan) return err(call.id, `no such tool: ${call.name}`);
  if (!plan.mutates) {
    const value = conRead(ctx, reg, call.name, plan.argv, plan.args);
    return { toolUseId: call.id, content: JSON.stringify(value), isError: false };
  }
  const outcome = runMutation(ctx, reg, call.name, plan.argv, plan.args, plan.
    reason);
  return outcome.ok
    ? { toolUseId: call.id, content: JSON.stringify(outcome.result), isError: false
     }
    : { toolUseId: call.id, content: `refused by harness: ${outcome.reason}`,
    isError: true };
}
```

Three properties hold at this seam. The model proposes; the harness disposes. The model names a capability and supplies arguments, and the harness decides whether that capability is in the registry and whether the reason is present. The decoder is the boundary. The conn returns untyped JSON, and the typed result the model sees is what the harness vouches for. The reason-gate is a smart constructor on the shapes the worker may run, which is the same role the fuel gate and the weapons-authorisation gate play in the star system.

In the many-worker sense the same seam is how a composed leaf would meet the harness. A Fighter leaf's answer function would call `dispatch` with an `attack` call, and `tensorC` above it would never see the gate at all. The orchestration composes positions; the harness licenses effects; the two meet only here.

7 One flag: deterministic or language model

The provider is chosen once, by a flag, in `src/maze-demo.ts`.

```
const live = Deno.args.includes("--live");
const client = live ? groqClient() : scriptedClient(wp);
```

Both values are a `ModelClient`, so the loop is identical in either mode.

7.1 Deterministic mode

`scriptedClient` in `src/agent/scripted.ts` is a breadth-first search. Each turn it reads the live world, finds a shortest route to the exit treating monsters as passable, and emits one tool call toward it: an `attack` when a monster stands in the next cell, otherwise a `move`. The search explores neighbours in a fixed order, so the sequence of tool calls is the same every run. It needs no key and no network.

Determinism rests on two seams the rest of `starship` already has (`src/log/ids.ts`). The id source and the clock are parameters, not ambient calls. The demo passes `seededIds` and `fixedClock`, which produce the same correlation ids and timestamps in the same order every run. Real use passes `realIds` and `realClock`. `verify.sh` runs the maze demo twice and diffs the standard output and both logs; they are byte-identical.

7.2 Language-model mode

`groqClient` in `src/agent/groq.ts` is the only file that knows a wire format. It posts to Groq's free, OpenAI-compatible chat-completions endpoint over plain `fetch`, with the tools in OpenAI function shape and `temperature` zero. It maps the model's `tool_calls` to `ToolCall` values and prior results to `tool` messages. Groq serves open-weight models for free. The catalogue changes over time, so the set a key can use is read from `GET /v1/models` rather than assumed. The endpoint shape is shared by other providers, so pointing `GROQ_BASE_URL` at Cerebras, OpenRouter, or a local Ollama at `http://localhost:11434/v1` switches the backend with no code change.

Environment variable	Meaning
`GROQ_API_KEY`	required for a live run; free from `console.groq.com`
`GROQ_MODEL`	default `openai/gpt-oss-120b`
`GROQ_BASE_URL`	default `https://api.groq.com/openai/v1`

The free tier throttles on tokens per minute, and the transcript grows every turn, so a rate-limit reply (HTTP 429) part way through a run is routine. `converse` waits it out and retries, honouring the `Retry-After` header and then the delay the body names, so a run completes on the free tier. It pauses rather than failing.

The gate is what makes a weaker free model safe rather than merely cheaper. A move with no reason is refused before it runs. A step into a wall is refused. A capability outside the registry cannot be named. Each refusal returns to the model as a result it can read and retry, so a weaker model burns steps rather than corrupting the world. Section 8 records a live run where exactly this happened.

```
# deterministic, no key, replays byte-identically
deno task maze

# a real free model drives the same loop through the same gate
```

```
export GROQ_API_KEY=gsk_...
deno task maze -- --live
```

8 Running and checking it

Command	What it does
`deno task maze`	the whole maze run: start map, the agent's reasoned steps, the solved map, both logs, reconciliation
`deno task maze -- --live`	the same, driven by a free model (needs `GROQ_API_KEY`)
`deno task dungeon -- look`	drive the maze CLI directly
`deno task test`	41 unit and contract tests, 17 of them the maze and loop
`deno task check`	type-check
`./verify.sh`	check, test, and prove both demos replay byte-identically

The scripted run reaches the exit in thirteen actions. It fights the goblin in one blow and the orc in two, the second of which costs the hero two hit points. The reconciliation reports thirteen matched mutations, one blocked order (a step into a wall, recorded only in the intent-log), and no alarms. The ship's log is the trace of what the agent did; the intent-log is why; together they are the solution, and reconciliation proves the agent moved the maze only through reasoned, logged orders.

Both modes have been run. The language-model mode was executed against Groq with the model `openai/gpt-oss-120b`. The model reached the exit in the same thirteen committed actions, over twenty-eight model turns, ending with eighteen of the hero's twenty hit points (the orc struck back once). On one turn it requested an attack on the goblin before the hero was adjacent; the harness refused the call; the model read the refusal, moved next to the goblin, and struck. The run crossed the free tier's tokens-per-minute limit several times and the retry waited each one out. The reconciliation was clean: thirteen matched, one blocked, no alarms, the same shape as the scripted run. The agent that solved the maze was a language model, and the gate held.